

AI 504 – Decidability of $\mathcal{A}$

Stefano Volpe

2026-05-06

We work with $\mathcal{A}$, the language of “All p are q ”, using Haskell. At the end of this problem set, we will have built a decider for derivability in $\mathcal{A}$. By soundness and completeness, this is the same as a decider for logical consequence in $\mathcal{A}$. We can refer to our program as simply a “decider for $\mathcal{A}$ ”.

Grading. The submitted code will be inspected and tested. A grade of 2 or higher means that you have submitted a compiling program declaring all identifiers you are asked to implement. On top of which, a 4 means that you have understood all questions. A 7 means a correct answer to at least half of the problems. A 10 means a correct answer to at least three fourths the problems. A 12 means a correct answer to all problems. Grade 0 is awarded on an ad-hoc basis. The total problem set grade will be an average of the programming grade and the written homework grade.

Instructions. All instructions from the parent problem set apply. Any way of receiving “intelligent” help via a computer is expressly forbidden. The following are the only allowed import statements: you can use any declaration imported via these.

```
1 import Data.Foldable
2 import Data.List.NonEmpty as NE
3 import Data.Set as S
```

Please paste a (corrected) version of your solution for homework 5 below these import statements. You will need part of it. You are always allowed to invoke previously defined functions (from the current homework, or the previous one).

Problem 1: Find a path by its starting vertex

Let v be a type representing the vertices of some graph G . Recall that we encode a path in said graph as a non-empty list of vertices using the `NonEmpty` type constructor. Define a binary curried function `findPathFrom` that takes a vertex a in G as first parameter, and a set of paths in G as second parameter¹. Your function should be parametric in v as a type variable, where v is assumed to implement `Ord`. If the second argument contains one or more paths whose starting vertex is a , the function should return one of these paths (it does not matter which). If no such paths are in the set, the function should still terminate, but without returning any paths whatsoever.

In Haskell, to express an optional return value, you can use the `Maybe` unary type constructor, which is already part of the prelude, and thus does not need to be explicitly imported or defined by you. Here is how it is defined:

```
1 data Maybe a = Nothing | Just a
```

Which function might help you find an element in a `Set`?

Example 1.1: Consider a graph whose set of vertices is $\mathbb{Z}$. Here is a possible correct result for `findPathFrom` (albeit not the only one!) on a specific input.

$$\text{findPathFrom}(3, \{1 \rightarrow 2 \rightarrow 3, 3 \rightarrow 2, 3 \rightarrow 1 \rightarrow 1\}) = 3 \rightarrow 1 \rightarrow 1$$

Returning path $3 \rightarrow 2$ would also have been correct.

¹As per usual, we use the `Set` type constructor to encode sets, even though it only supports finite sets.

```
1 -- >>> findPathFrom 3 (S.fromList [1 :| [2, 3], 3 :| [2], 3 :| [1, 1]])
2 -- Just (3 :| [1,1])
```

In the next example, the function cannot return any path at all. In mathematics, we sometimes indicate this by returning a special constant $\perp$ which is distinct from any other path.

Example 1.2: Consider a graph whose set of vertices is $\mathbb{Z}$. Here is a possible correct result for `findPathFrom` (albeit not the only one!) on a specific input.

$$\text{findPathFrom}(9, \{1 \rightarrow 2 \rightarrow 3, 3 \rightarrow 2, 3 \rightarrow 1 \rightarrow 1\}) = \perp$$

```
1 -- >>> findPathFrom 9 (S.fromList [1 :| [2, 3], 3 :| [2], 3 :| [1, 1]])
2 -- Nothing
```

Problem 2: Convert a path to a proof tree

Consider a fixed set of nouns P . Then, a path in the all-graph of a set of sentences corresponds to a sequence of sentences in that set.

Example 2.1: Fix `Char` as the set of nouns. If $'a' \rightarrow 'b' \rightarrow 'c' \rightarrow 'd'$ is a path in the all-graph of a set of sentences Γ , then

All $'a'$ are $'b'$, All $'b'$ are $'c'$, All $'c'$ are $'d'$ $\in \Gamma$.

Starting from this observation, come up with a way to convert a path from a to b in the all-graph of a set of sentences Γ into a proof tree with root at “All a are b ” over Γ . Implement this conversion as a function `pathToTree` which is parametric in an arbitrary type p representing the set of nouns. The function accepts a path in the all-graph of some set of sentences using nouns from p , and returns a correct proof tree according to whatever conversion strategy you came up with. There are many (in fact, infinite!) possible correct answers, and the results of your conversion need not be as small as possible.

To test your solution, you should add the following deriving declaration to your program.

```
1 deriving instance (Eq p) => Eq (ProofTree p)
```

Whenever a certain type p implements typeclass `Eq`, this ensures Haskell derives the “reasonable” implementation of `Eq` for `ProofTree p`, as well. This is just the usual notion of equality for trees with ordered children.

Example 2.2: Fix $\mathbb{Z}$ as a set of nouns. Here is a possible correct conversion (albeit not the only one!) on a specific input.

$$\text{pathToTree}(1 \rightarrow 2 \rightarrow 3 \rightarrow 4) = \frac{\frac{\text{All 1 are 2} \quad \text{All 2 are 3}}{\text{All 1 are 3}} \text{Barbara} \quad \text{All 3 are 4}}{\text{All 1 are 4}} \text{Barbara}$$

```
1 -- >>> pathToTree (1 :| [2, 3, 4]) == Barbara (Barbara (Assume 1 2) (Assume 2 3)) (Assume 3 4)
2 -- True
```

Problem 3: The canonical model of a set Γ

In this problem, we implement a function `canMod` computing the canonical model of a set Γ as seen in Definition 1.26. The function should be parametric in a type $\mathfrak{p}$, which represents the set of nouns and is assumed to implement `Ord`. To allow for an implementation which is closer to the pseudocode of Algorithm 1.32 in the next problem, the input to your function should not be Γ itself, as implied by Definition 1.26. Rather, the only input should itself be a function². This input function is assumed to map each noun u to the set of all possible paths ending in u in the all-graph of Γ . When there are more than one such paths sharing the same starting vertex, the input function is only assumed to include at least one of them in its return value.

Example 3.1: Let $P = \mathbb{Z}$, $\Gamma = \{\text{All } 1 \text{ are } 2, \text{All } 2 \text{ are } 3\}$. The all-graph of Γ is

$$1 \longrightarrow 2 \longrightarrow 3$$

where the vertices with no incoming or outgoing edges are not shown. Then, the following function satisfies our assumptions on the input.

$$f(z) = \begin{cases} \{1 \rightarrow 2, 2\} & \text{if } z = 2 \\ \{1 \rightarrow 2 \rightarrow 3, 2 \rightarrow 3, 3\} & \text{if } z = 3 \\ \{z\} & \text{otherwise} \end{cases}$$

```
1 f :: Int -> Set (NonEmpty Int)
2 f 2 = S.fromList [1 :| [2], 2 :| [] ]
3 f 3 = S.fromList [1 :| [2, 3], 2 :| [3], 3 :| []]
4 f z = S.singleton (z :| [])
```

We can use it to compute the canonical model of Γ .

$$\text{canMod}(f) = (\mathbb{Z}, \llbracket \cdot \rrbracket : \mathbb{Z} \rightarrow \mathcal{P}(\mathbb{Z}))$$

$$\llbracket z \rrbracket = \begin{cases} \{1, 2\} & \text{if } z = 2 \\ \{1, 2, 3\} & \text{if } z = 3 \\ \{z\} & \text{otherwise} \end{cases}$$

So we have $\text{canMod}(f) \models \text{All } 1 \text{ are } 3$ and $\text{canMod}(f) \not\models \text{All } 2 \text{ are } 1$.

```
1 -- >>> canMod f |= AllAre 1 3
2 -- True
3
4 -- >>> canMod f |= AllAre 2 1
5 -- False
```

Problem 4: A decider for $\mathcal{A}$

For this problem, you will need to use functions you already defined in this homework and in the previous one. You must implement a decider for $\mathcal{A}$ parametric in a type variable s which represents

²Functions whose argument and/or return value might also be a function are said to be *higher-order*. For example, `map`, `fold`, and `filter` all accept functions as arguments, and are thus higher-order. Similarly, all curried functions you have implemented so far return a function when they are invoked with only one argument, so they are higher-order as well.

Higher-order functions contribute to making functions *first-class citizens* of our programming language: every language feature (in this case, “being a parameter” and “being a return value”) should be available to them. While not everybody agrees on what functional programming is, many think having functions as first-class citizens is certainly one of the ingredients.

the set of nouns and that is assumed to implement `Ord`. The decider should be a binary curried function named `decide` that implements Algorithm 1.32. Its first argument is Γ , a set of sentences with nouns from s . Its second argument is φ , a sentence with nouns from s . If $\Gamma \vdash \varphi$, the function should return a derivation tree witnessing this fact. Otherwise, the function should return the canonical model $\mathcal{M}_\Gamma$ of Γ (refuting $\Gamma \vdash \varphi$ is equivalent to refuting $\Gamma \models \varphi$, and if the latter does not hold, then it is the case that $\mathcal{M}_\Gamma \models \Gamma$ and $\mathcal{M}_\Gamma \not\models \varphi$, and this is sufficient evidence). To allow for a return value which is of either type, we use the following binary type constructor.

```
1 data Either a b = Left a | Right b
```

It is already part of the prelude, so it does not need to be explicitly imported or defined by you. You must use the `Left` term constructor when returning a proof tree, and the `Right` term constructor when returning the canonical model.

Recall that, in problem 4.6 of the last homework, we chose to represent the reflexive-transitive closure of a graph via a function able to compute all paths ending in a given vertex.

Example 4.1: Consider the values of P and Γ from the previous example. Since $\Gamma \vdash$ All 1 are 3, then one correct output for a specific input is

$$\text{decide}(\Gamma, \text{All 1 are 3}) = \frac{\text{All 1 are 2} \quad \text{All 2 are 3}}{\text{All 1 are 3}} \text{Barbara},$$

although any other proof tree witnessing $\Gamma \vdash$ All 1 are 3 would also have been a correct output.

```
1 -- >>> either (== Barbara (Assume 1 2) (Assume 2 3)) (const False) (decide (S.fromList [AllAre 1 2, AllAre 2 3]) (AllAre 1 3))
2 -- True
```

We now move to a non-derivability example.

Example 4.2: Consider again the same values of P and Γ . Since $\Gamma \not\vdash$ All 2 are 1, then every correct implementation must satisfy

$$\text{decide}(\Gamma, \text{All 2 are 1}) = \mathcal{M}_\Gamma,$$

where $\mathcal{M}_\Gamma$ is the canonical model of Γ , so

$$\mathcal{M}_\Gamma \models \Gamma \quad \text{and} \quad \mathcal{M}_\Gamma \not\models \text{All 2 are 1}.$$

```
1 -- >>> either (const False) (\m -> m == AllAre 1 2 && m == AllAre 2 3 && not (m == AllAre 2 1)) (decide (S.fromList [AllAre 1 2, AllAre 2 3]) (AllAre 2 1))
2 -- True
```

We now mention some Haskell features that can help your implementation of Algorithm 1.32 look similar to the pseudocode found in the book, if you wish so. Using these features can be helpful, but is entirely optional for this homework.

4.1 let bindings

Lines 2 and 3 of the algorithm in the pseudocode are `let` bindings. These allow for variable declarations within an expression.

<https://learnyouahaskell.github.io/syntax-in-functions.html#let-it-be>

4.2 Case expressions

Line 4 of the algorithm in the pseudocode is presented as a simple conditional, but this is very misleading. In the `then`-branch, the computation requires starting from a path from p to q in the all-

graph which witnesses the condition from line 4 being satisfied. Therefore, evaluating the condition should not return a simple boolean value, but rather an optional path (recall that we model optional values using the `Maybe` type constructor seen above), on which we can then perform pattern matching to branch. Pattern-matching within expressions is achieved via case expressions.

<https://learnyouahaskell.github.io/syntax-in-functions.html#case-expressions>